

바이트 코딩

• 왜 프롬프트가 중요한가?

> 같은 AI, 다른 결과

- 똑같은 클로드를 사용해도 프롬프트(질문/요청)를 어떻게 쓰느냐에 따라 답변의 품질과 방향이 완전히 달라집니다.

> 바이브 코딩에서는 더 중요하다

- AI에게 코드를 대신 짜 달라고 요청하는 것이기 때문에, 무엇을 어떻게 요청하느냐가 곧 결과물의 완성도를 결정합니다.

나쁜 예

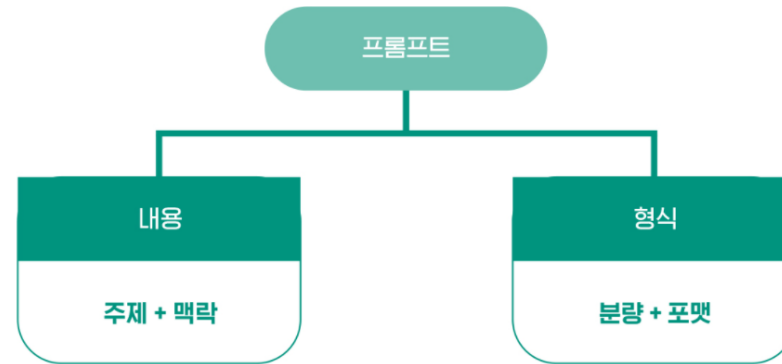
"웹사이트 만들어줘"

좋은 예

"카페 소개 웹사이트를 HTML/CSS로 만들어줘. 소개, 메뉴, 오시는 길 3개 섹션으로 구성해줘"

• 프롬프트의 기본 구조 5요소

> 프롬프트의 기본 구조



> 좋은 프롬프트는 다음 특성을 담고 있다

- 역할 전달
- 맥락 전달
- 목적 전달
- 출력 형식 전달

• 프롬프트 사용법

> 역할 전달

- ChatGPT에 역할을 부여하면, 그 역할의 관점과 지식을 바탕으로 응답하기 때문에 사용자가 원하는 정보와 어조에 더 가까운 답변을 얻을 수 있다.

"너는 10년차 파이썬 백엔드 개발자야. 이 코드를 리뷰해줘."

> 맥락 전달

- 질문의 배경과 상황에 대한 정보가 부족하면 추측을 통해 답변을 생성하기 때문에 의도하지 않은 결과가 출력될 가능성이 높아진다.

"이 프로젝트는 Python 3.13, Flask로 만든 웹앱이야. 이 상황에서 로그인 기능을 추가하고 싶어."

• 프롬프트 사용법

> 목적 전달

- 질문의 목적을 명확하게 전달하면 ChatGPT가 원하는 바를 정확히 이해하고 답변을 생성할 수 있다.

목적	예시 명령어
설명이 필요할 때	~에 대해 초보자도 이해할 수 있게 설명해줘
코드 생성이 필요할 때	~하는 기능을 하는 코드를 작성해줘
오류 해결이 필요할 때	이 에러 메시지의 원인과 해결 방법을 알려줘

> 출력 형식 전달

- 일반적으로 ChatGPT의 대답은 텍스트 형식이다. 하지만 표, 리스트 등 다양한 포맷형식의 데이터가 필요하다면 프롬프트로 요청하면 된다.

"결과를 표로 정리해줘"

"코드만 코드 블록으로 작성해줘"

"단계별 번호 목록으로 정리해줘"

• 바이브 코딩 실전 프롬프트 예시

> 요소를 모두 담은 프롬프트

- 역할, 맥락, 목적, 출력 형식을 한 번에 담아 요청하면 훨씬 완성도 높은 코드를 받을 수 있다.

"너는 숙련된 프론트엔드 개발자야. 나는 코딩 초보자이고,
할 일 목록을 관리하는 웹페이지가 필요해.
항목 추가·삭제·완료 체크 기능을 포함한 HTML/CSS/JS 코드를 작성해주고,
각 파일이 어떤 역할을 하는지 주석으로 설명도 남겨줘."

- 프롬프트 사용 시 주의사항

- > 한 번에 한 가지 주제만 이야기하기

- 주제가 바뀔 때는 새 대화를 시작해야 더 좋은 답변을 받을 수 있다.

- > 결과가 만족스럽지 않다면 다르게 다시 물어보기

- 이번 장에서 배운 5가지 요소를 다시 점검하며 질문을 구체화한다.

- > AI의 답변을 그대로 믿지 않고 검증하기

- AI가 만든 코드도 항상 정확하지는 않으므로, 실행해보고 결과를 확인하는 습관이 필요하다.

• 클로드(Claude)란?

> Anthropic 회사가 만든 생성형 AI

- 대화, 글쓰기, 코드 작성, 파일 분석 등 다양한 작업을 도와주는 AI 어시스턴트

> 이 수업에서 클로드를 사용하는 이유

- 긴 대화 맥락 이해, 코드 작성/설명 능력이 뛰어나 바이브 코딩에 적합

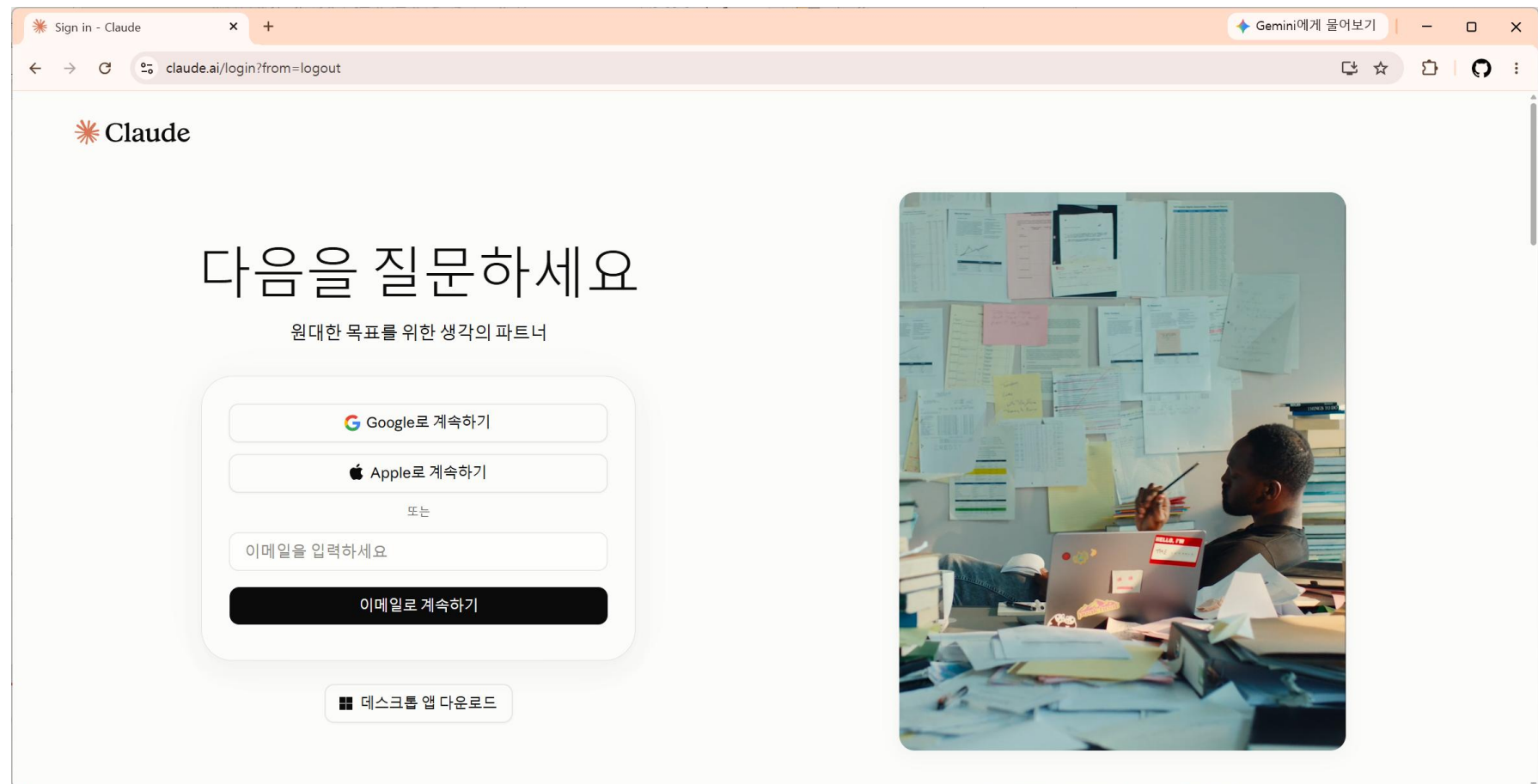
4대 생성형 AI 전격 비교
(COMPARISON OF 4 GENERATIVE AIs)

비교 항목 (Characteristics)	ChatGPT	Grok	Gemini	Claude
개발사 (Developer)	ChatGPT	Grok-2	Gemini	Claude
핵심 기능 (Key Features)	o3, GPT-4o	실시간 정보, X 연동, 'Ani' 컴패니언	Gemini Ultra, Advanced	Claude 3 Opus, 3.5 Sonnet
강점 (Strengths)	문제 해결, 코딩, 범용성	유머, 실시간 뉴스, 이미지 생성	영상 분석, 다국어 이해, 구글 연동	코딩 능력, 문장 구사, 작업 편의성
단점 (Weaknesses)	환각, 아침 성향	기능 제한, 뉴스 출처 무작위	컨텍스트 저하, 미국 중심 편향	배경 지식 부족, 집중 제한

- Claude 접속하고 로그인하기

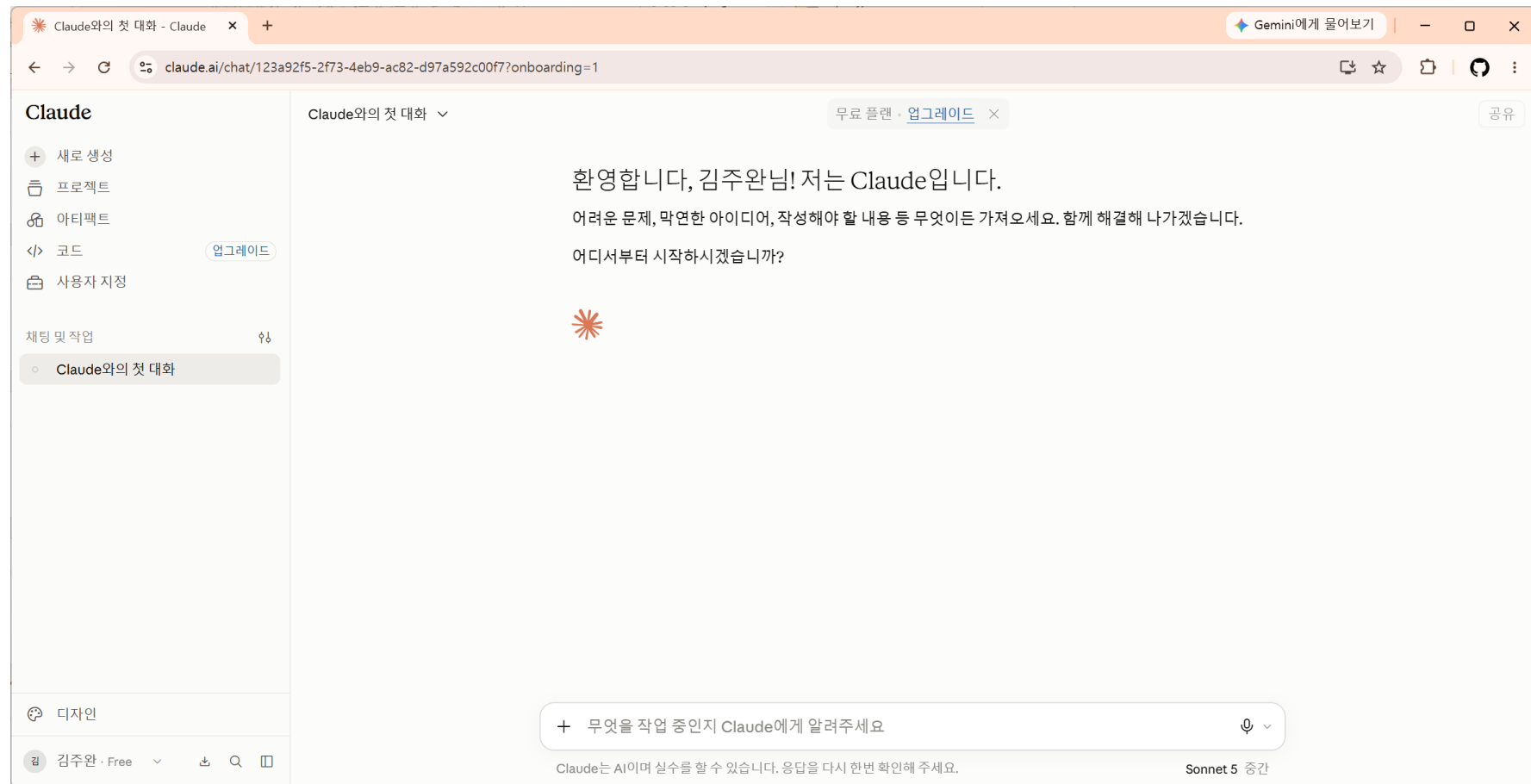
- > claude.ai 접속

- 구글 계정이 있으면 별도 가입 없이 간편하게 로그인할 수 있습니다.



- 클로드 화면 살펴보기

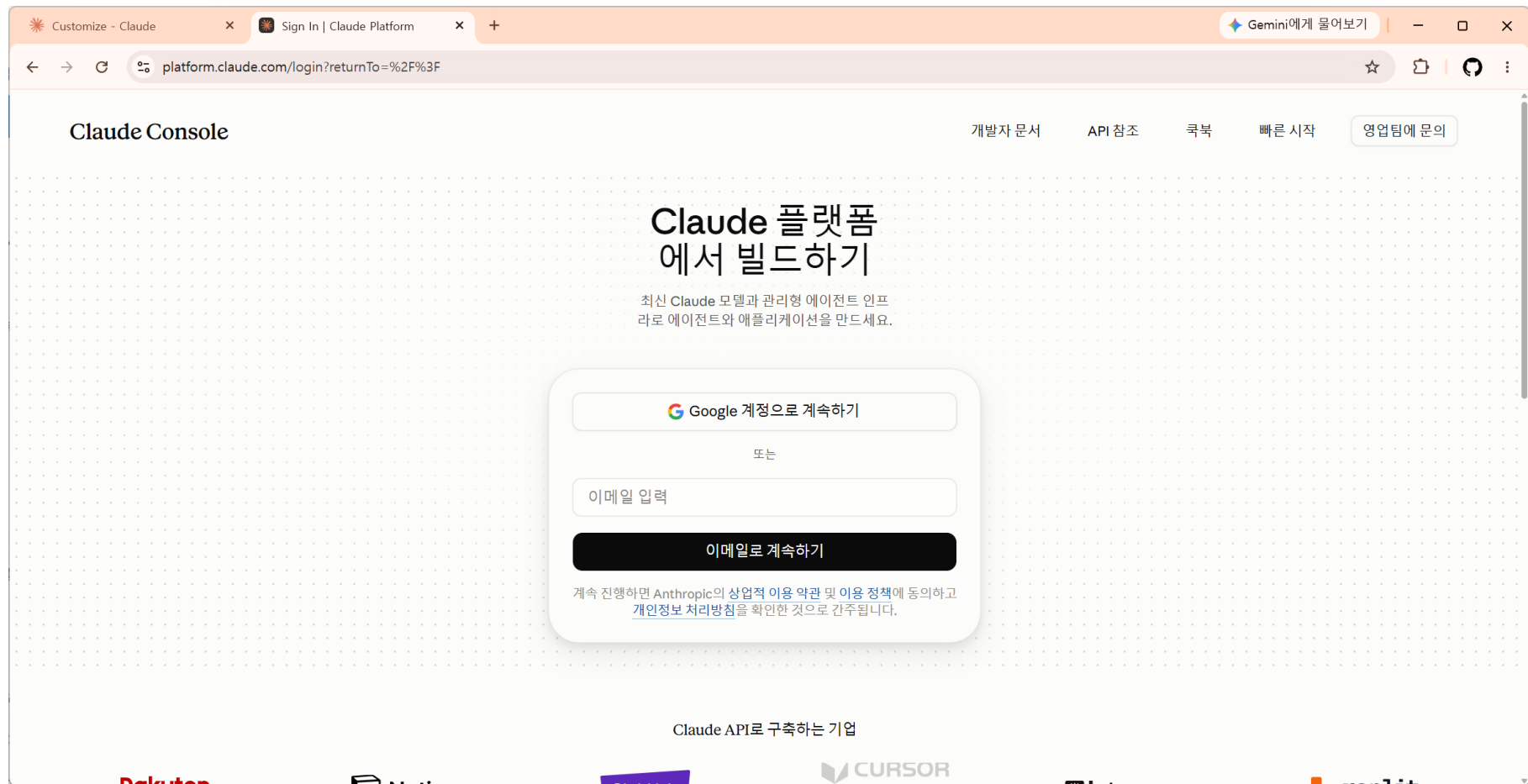
- > 기본 구성 익히기



• 클로드 API란?

> 내 프로그램 안에서 클로드를 호출하는 방법

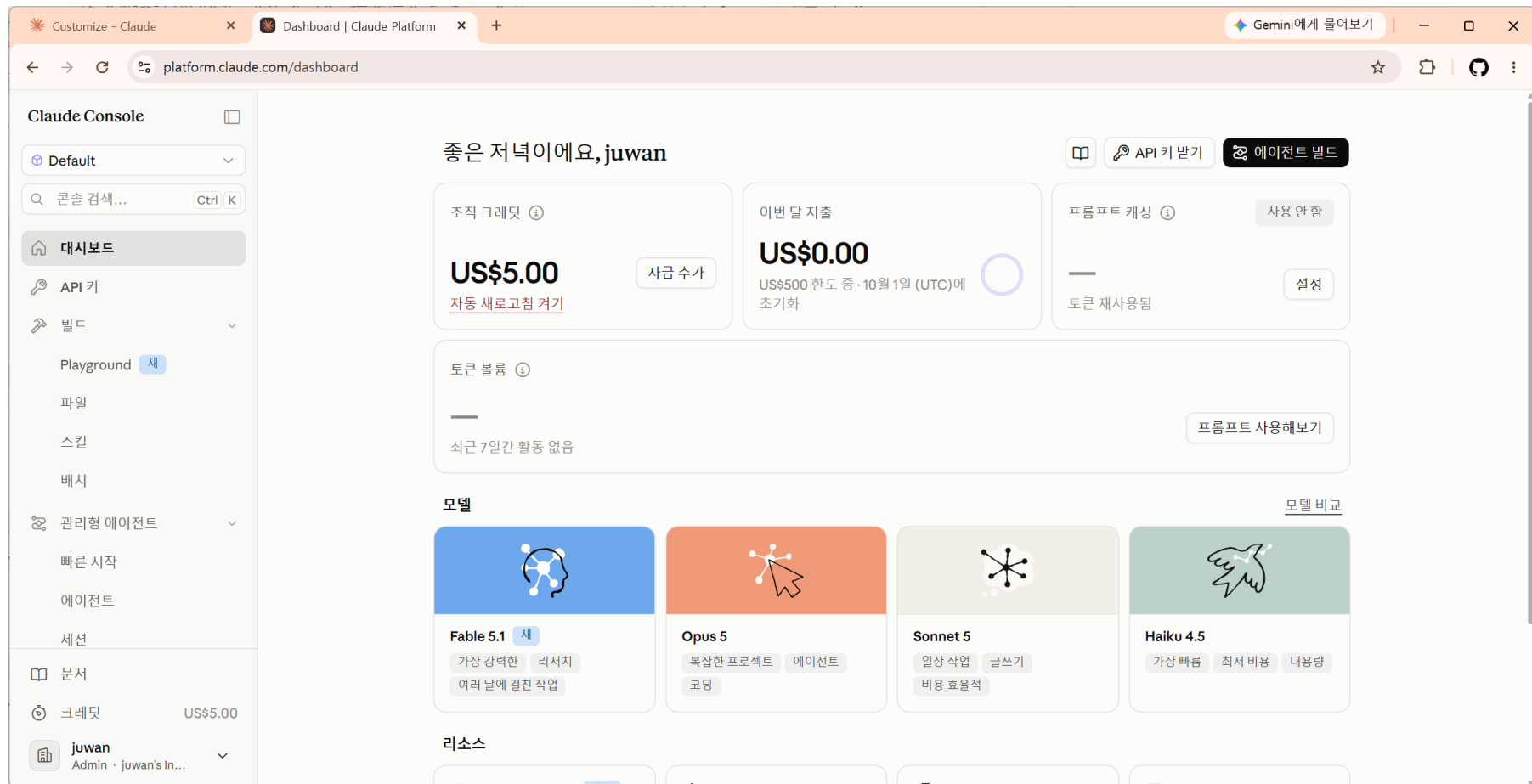
- 웹사이트에 접속하지 않고도, 코드에서 직접 클로드에게 질문하고 답을 받아올 수 있습니다.



• API 키 발급받기

> 클로드 플랫폼 접속

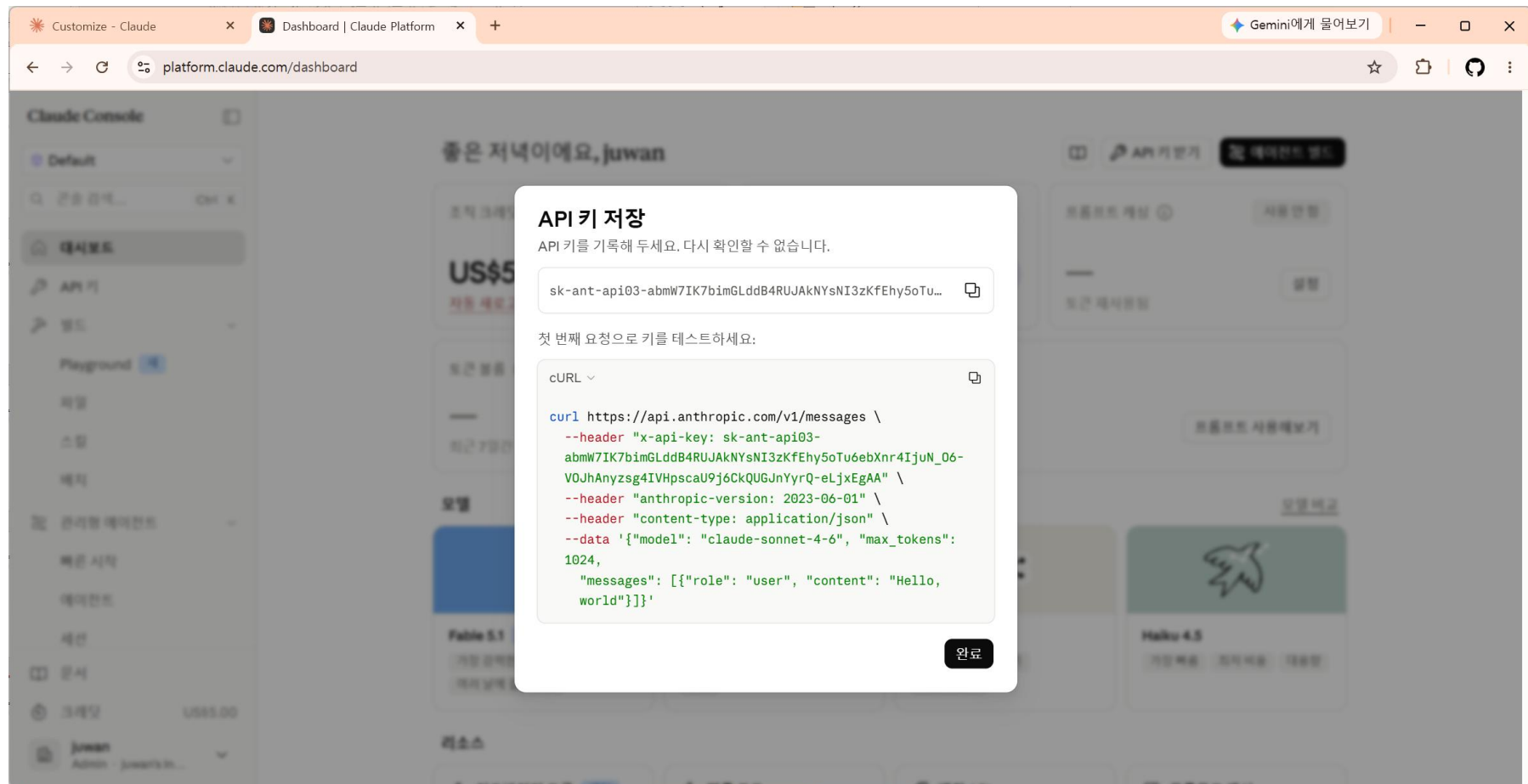
- 로그인 후 좌측 메뉴에서 API Keys 선택 → Create Key 로 새 키를 발급받습니다.



• API 키 발급받기

> 클로드 플랫폼 접속

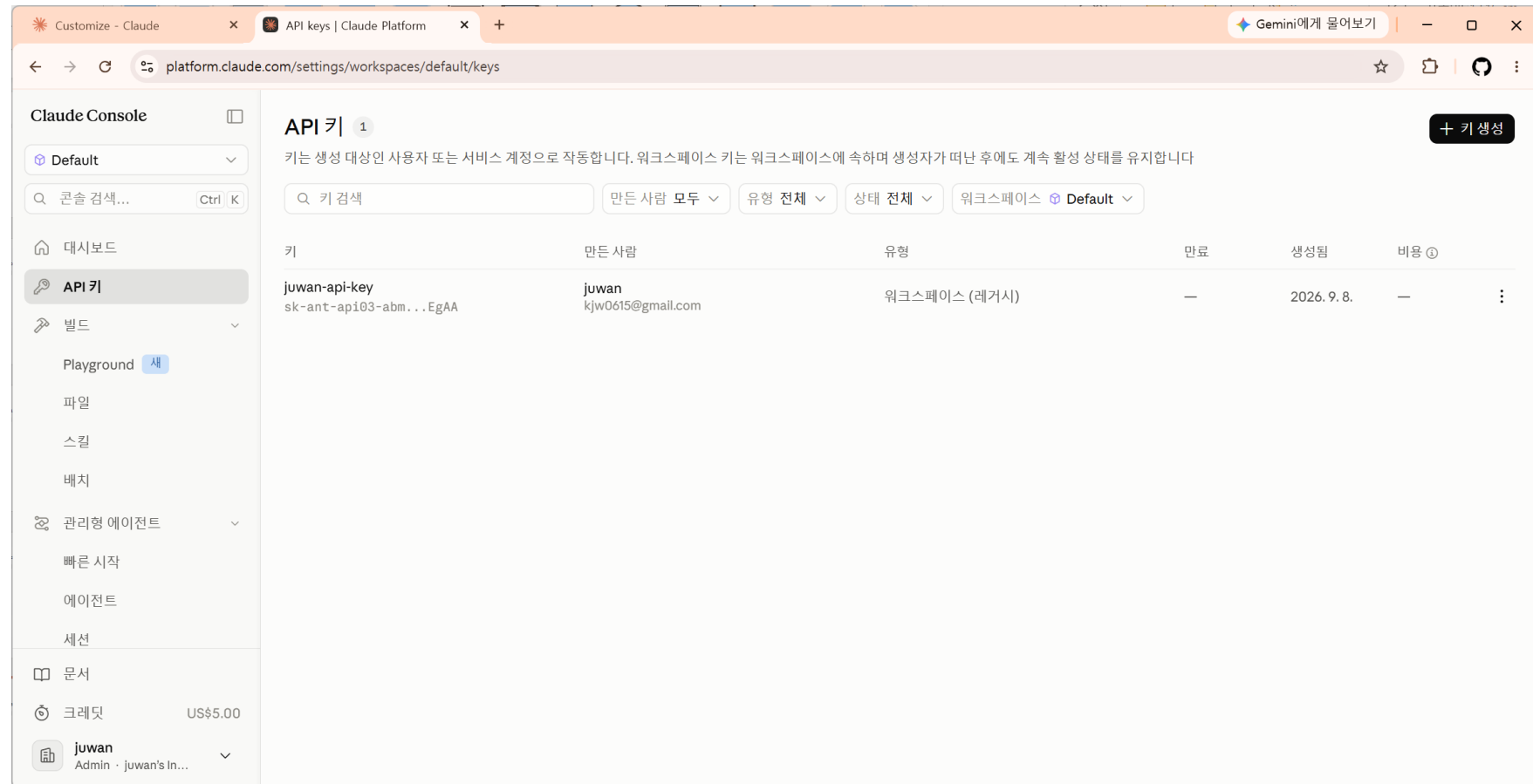
- 로그인 후 좌측 메뉴에서 API Keys 선택 → Create Key 로 새 키를 발급받습니다.



• API 키 발급받기

> 클로드 플랫폼 접속

- 로그인 후 좌측 메뉴에서 API Keys 선택 → Create Key 로 새 키를 발급받습니다.



- API 키 보안 주의사항

- > API 키는 비밀번호와 같다

- 화면 캡처, 채팅, 코드 공유 등 어디에도 그대로 노출하면 안 됩니다.

주의: 키가 노출되었다면 즉시 재발급하고, 코드에는 직접 적지 말고 환경 변수로 관리하세요.

- > 수업에 사용할 키

```
sk-ant-api03-4n3Ls-  
dyakWRX7YdiQKRFkFg_gbMpRA_pWE7DE_N8sgsnL3476y2pXpgQj59poTzIWLYyAeQehvQ_  
XaqV6_Hdg-pQe9uAAA
```

- 파이썬에서 Claude API 사용하기

> anthropic 라이브러리 설치 후 호출

16

```
pip install anthropic
```

```
import anthropic

client = anthropic.Anthropic(api_key="여기에 API 키")

message = client.messages.create(
    model="claude-sonnet-5",
    max_tokens=1024,
    messages=[
        {"role": "user", "content": "너는 누구야?"}
    ]
)

print(message.content[0].text)
```

• 주요 매개변수 이해하기

> 요청 시 자주 사용하는 옵션

매개변수	필수/선택	설명
model	필수	사용할 모델을 지정 (예: claude-sonnet-4-5)
messages	필수	대화 내용(user/assistant 메시지)을 입력
max_tokens	필수	생성할 답변의 최대 길이를 지정
Temperature	선택	0~1 사이 값. 낮을수록 일관된 답변, 높을수록 창의적인 답변
system	선택	시스템 프롬프트(모델의 역할, 지침 등)

```
message = client.messages.create(  
    model="claude-sonnet-5",  
    max_tokens=1024,  
    system="당신은 친절한 여행 가이드입니다.",  
    messages=[  
        {"role": "user", "content": "교토 여행 코스 추천해줘"},  
        {"role": "assistant", "content": "네, 교토는..."},  
        {"role": "user", "content": "3일 코스로 다시 짜줘"}  
    ]  
)
```

- Git이란? 왜 배워야 하는가

- > 코드의 변경 이력을 관리하는 도구

- 언제, 무엇을, 왜 바꿨는지 기록하고, 필요하면 이전 상태로 되돌릴 수 있습니다.

- > 바이브 코딩에서 Git이 필요한 이유

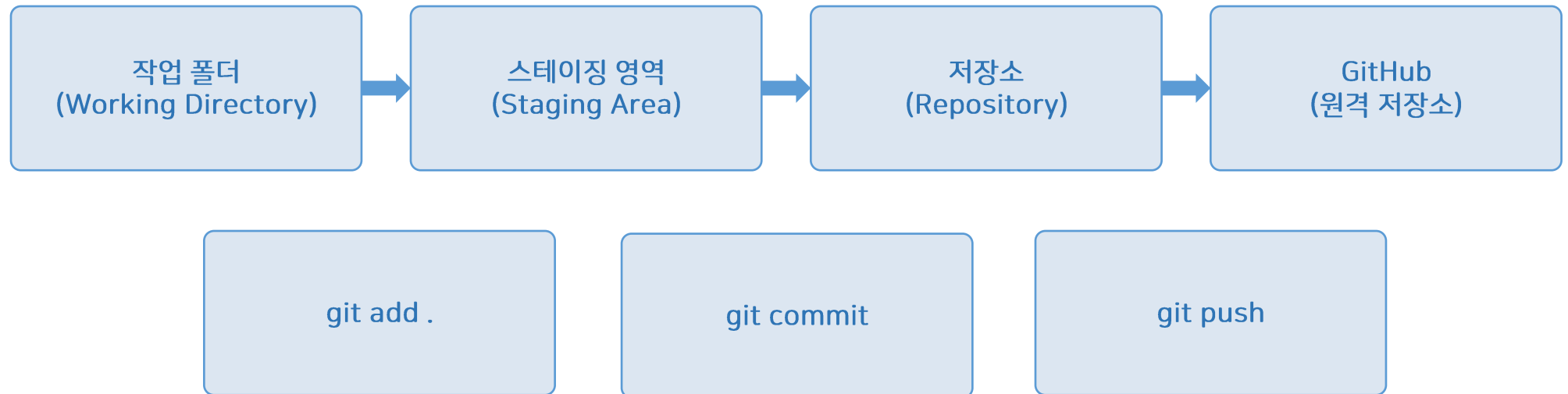
- AI가 코드를 계속 수정해주기 때문에, 잘 되던 버전으로 되돌릴 수 있는 안전장치가 꼭 필요합니다.

• Git 핵심 개념

> 코드가 이동하는 4단계

- 작업 폴더에서 수정한 내용을 커밋으로 기록하고, 원격 저장소(GitHub)에 올리는 흐름입니다.

19



• 바이브 코딩 방식으로 Git 배우기

> 모르는 명령어는 바로 클로드에게 물어보기

- 외우려 하지 말고, 상황을 설명하며 필요한 명령어를 그때그때 물어보는 것이 바이브 코딩 학습법입니다.

"지금 만든 프로젝트 폴더를 Git으로 관리하고 싶어. 초기 설정부터 순서대로 알려줘."

지금 만든 프로젝트 폴더를 Git으로 관리하고 싶어. 초기 설정부터 순서대로 알려줘.

지금 ↺ ✎ 🗑

좋아요! Git 초기 설정부터 원격 저장소 연결까지 순서대로 정리해드릴게요.

1 Git 설치 확인

터미널에서 `git --version` 을 입력해 설치 여부를 확인하세요. 설치가 안 되어 있다면 macOS는 `brew install git`, Windows는 `git-scm.com`에서 설치 파일을 받으면 됩니다.

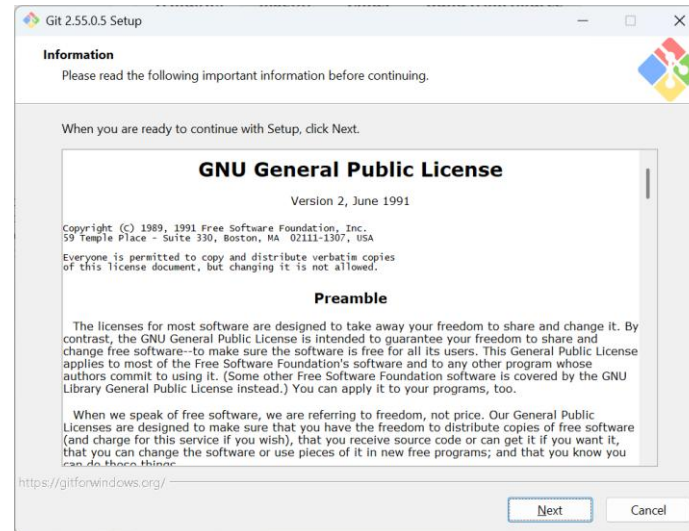
2 사용자 정보 등록 (최초 1회)

`git config --global user.name "이름"` 과 `git config --global user.email "이메일"` 을 입력해 커밋에 기록될 정보를 설정하세요. 컴퓨터 전체에 한 번만 하면 됩니다.

- 실습 환경 준비하기

- > Git 설치

- Git-2.55.0.5-64-bit 로 설치



- > Git 설치 확인

- VS Code 터미널을 열고 git --version 명령으로 설치 여부를 확인합니다.

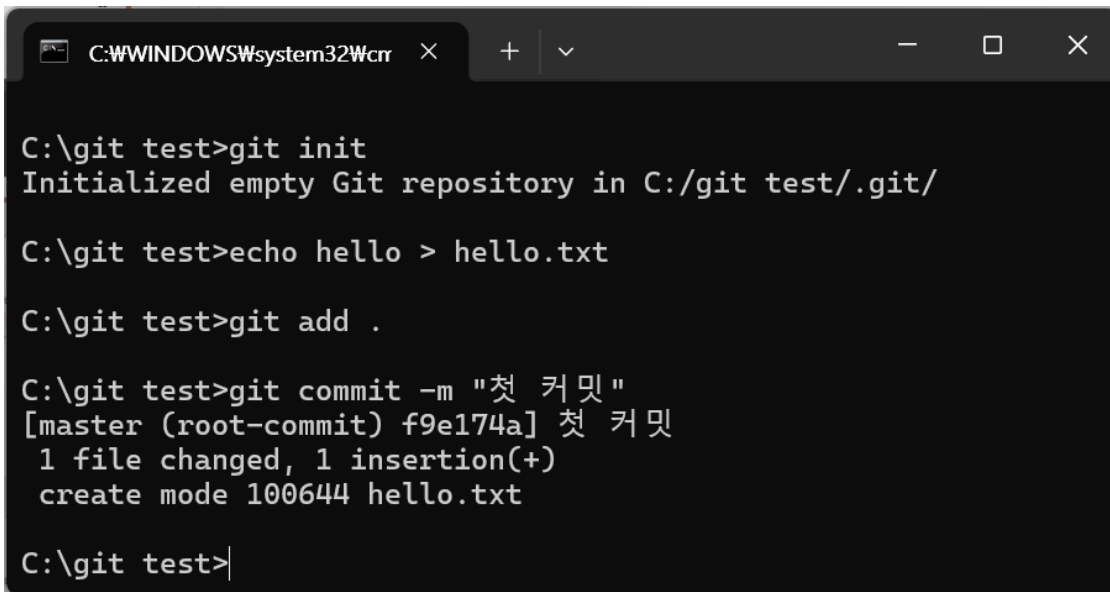
```
git --version
```

• 저장소 초기화하고 커밋하기 - 명령어

> 가장 기본이 되는 3가지 명령어

- 폴더를 저장소로 만들고, 변경 내용을 기록으로 남기는 과정입니다.

git init	# 현재 폴더를 Git 저장소로 초기화
git add .	# 변경된 모든 파일을 스테이징
git commit -m "첫 커밋"	# 커밋 메시지와 함께 기록 남기기



```
C:\WINDOWS\system32\cmd.exe
C:\git test>git init
Initialized empty Git repository in C:/git test/.git/

C:\git test>echo hello > hello.txt

C:\git test>git add .

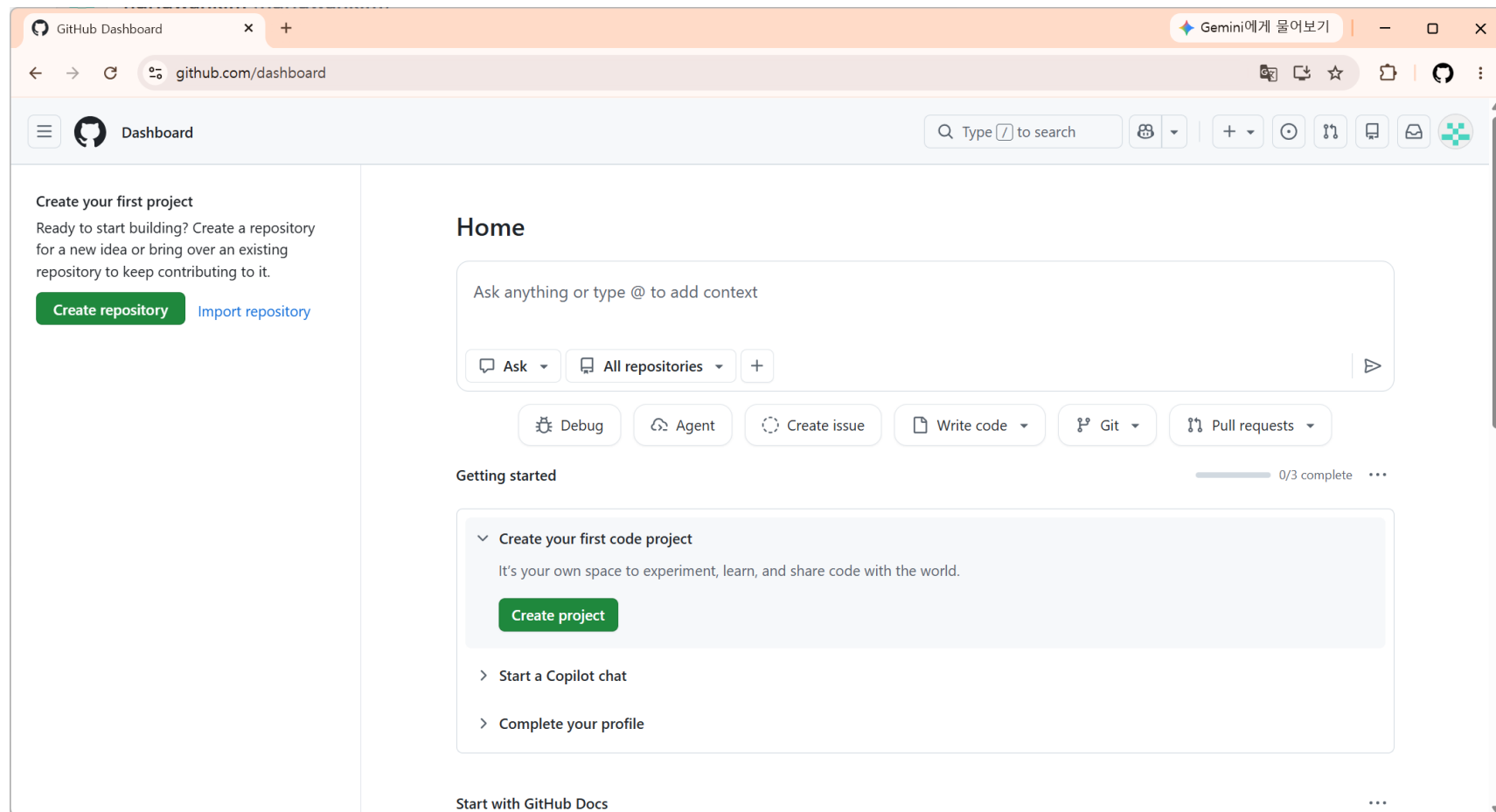
C:\git test>git commit -m "첫 커밋"
[master (root-commit) f9e174a] 첫 커밋
 1 file changed, 1 insertion(+)
 create mode 100644 hello.txt

C:\git test>
```

- GitHub에 원격 저장소 만들기

- > github.com에서 New repository

- 저장소 이름을 정하고 생성하면, 코드를 올릴 수 있는 온라인 주소가 만들어집니다.



- GitHub에 원격 저장소 만들기

- > github.com에서 New repository

- 저장소 이름을 정하고 생성하면, 코드를 올릴 수 있는 온라인 주소가 만들어집니다.

New repository

github.com/new

Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).

Required fields are marked with an asterisk (*).

1 General

Owner * hgrjuwankim / Repository name * git-test

git-test is available.

Great repository names are short and memorable. How about [fuzzy-computing-machine](#)?

Description

0 / 350 characters

2 Configuration

Choose visibility * Public

Choose who can see and commit to this repository

Add README Off

READMEs can be used as longer descriptions. [About READMEs](#)

Add .gitignore No .gitignore

.gitignore tells git which files not to track. [About ignoring files](#)

Add license No license

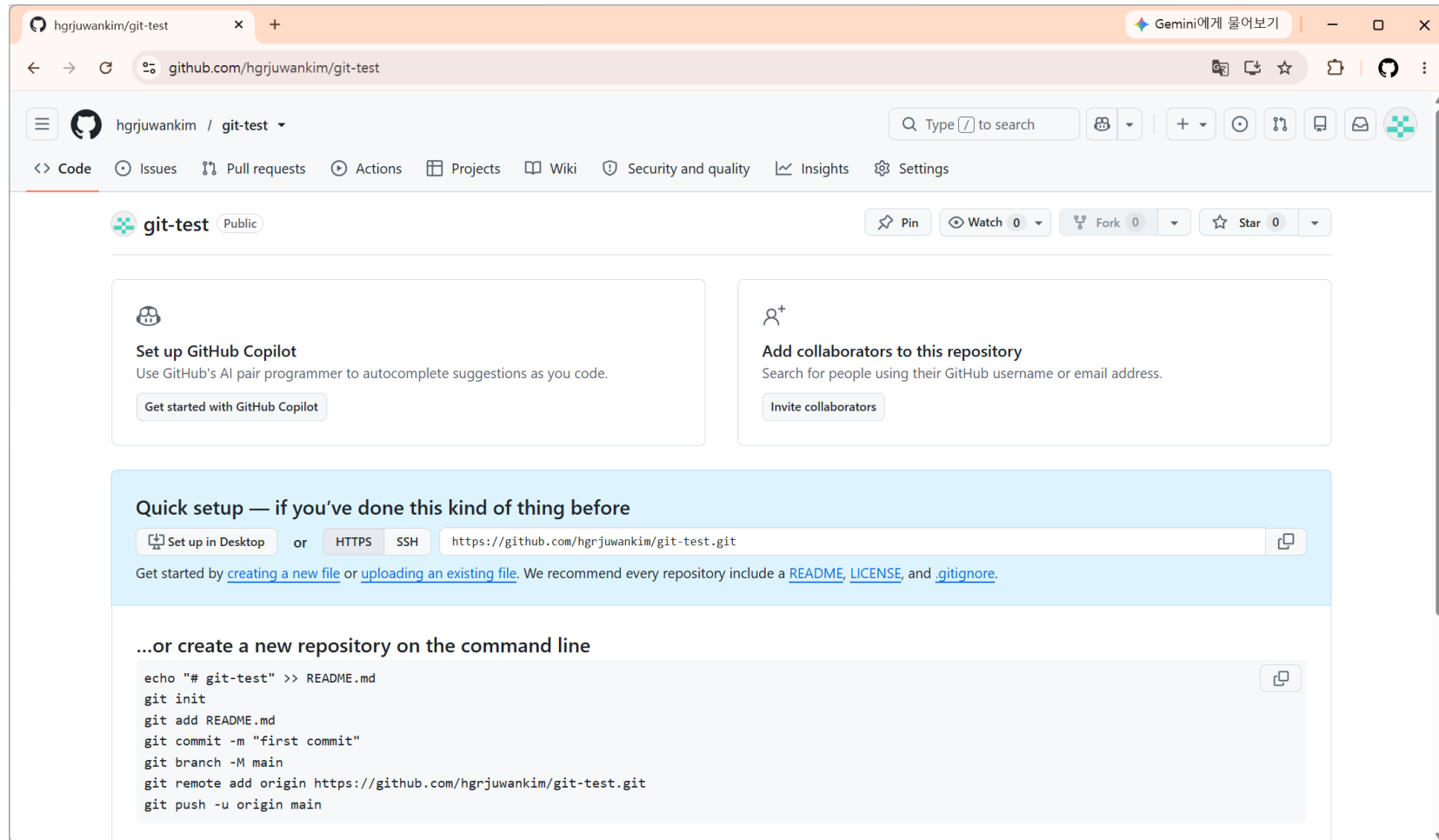
Licenses explain how others can use your code. [About licenses](#)

Create repository

• GitHub에 원격 저장소 만들기

> github.com에서 New repository

- 저장소 이름을 정하고 생성하면, 코드를 올릴 수 있는 온라인 주소가 만들어집니다.

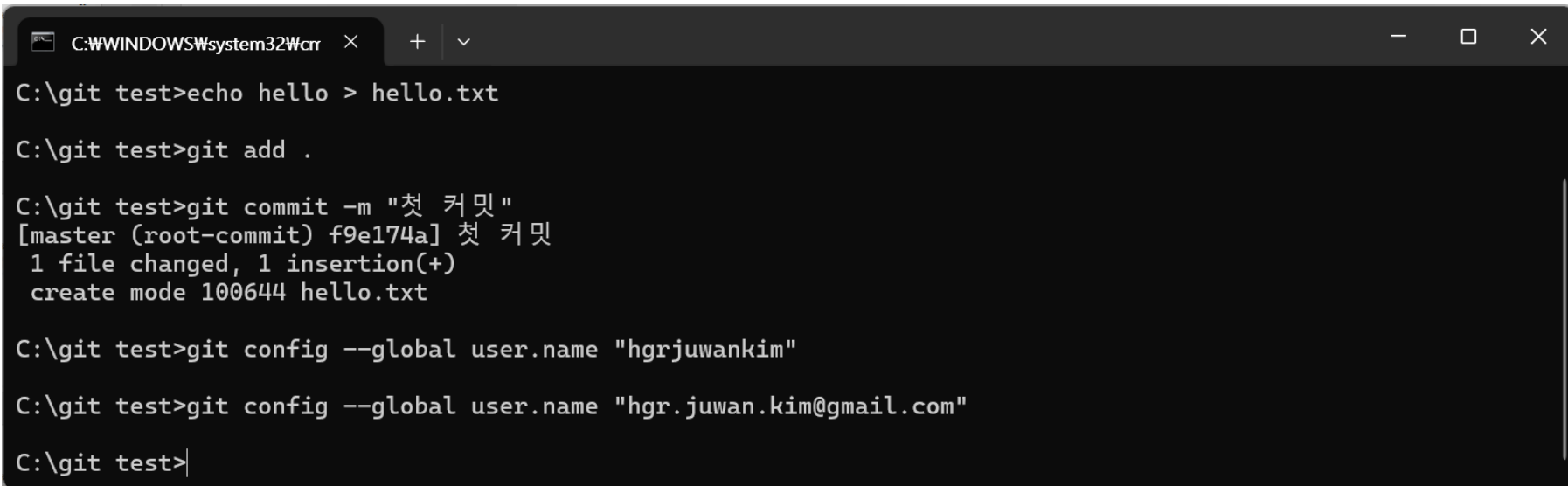


• 사용자 정보 등록

> 커밋에 기록될 정보 설정

- git config --global user.name "이름"
- git config --global user.email "이메일주소"

```
git config --global user.name "hgrjuwankim"  
git config --global user.email "hgr.juwan.kim@gmail.com"
```



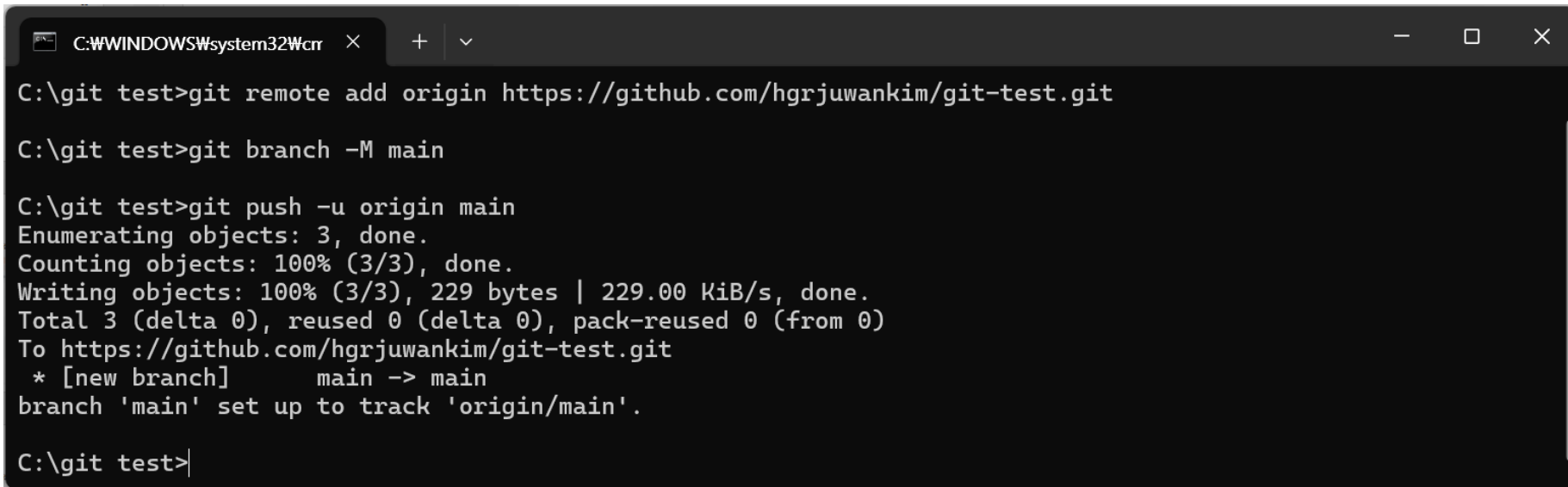
```
C:\WINDOWS\system32\cmd.exe
C:\git test>echo hello > hello.txt
C:\git test>git add .
C:\git test>git commit -m "첫 커밋"
[master (root-commit) f9e174a] 첫 커밋
1 file changed, 1 insertion(+)
create mode 100644 hello.txt
C:\git test>git config --global user.name "hgrjuwankim"
C:\git test>git config --global user.email "hgr.juwan.kim@gmail.com"
C:\git test>
```

• 로컬 저장소를 GitHub에 연결하고 올리기

> 원격 주소 연결 후 업로드(push)

- GitHub 저장소 페이지에서 복사한 주소를 아래처럼 연결합니다.
- `git remote add origin https://github.com/아이디/저장소이름.git`

```
git remote add origin https://github.com/hgrjuwankim/git-test.git
git branch -M main
git push -u origin main
```



```
C:\WINDOWS\system32\cmd.exe
C:\git test>git remote add origin https://github.com/hgrjuwankim/git-test.git

C:\git test>git branch -M main

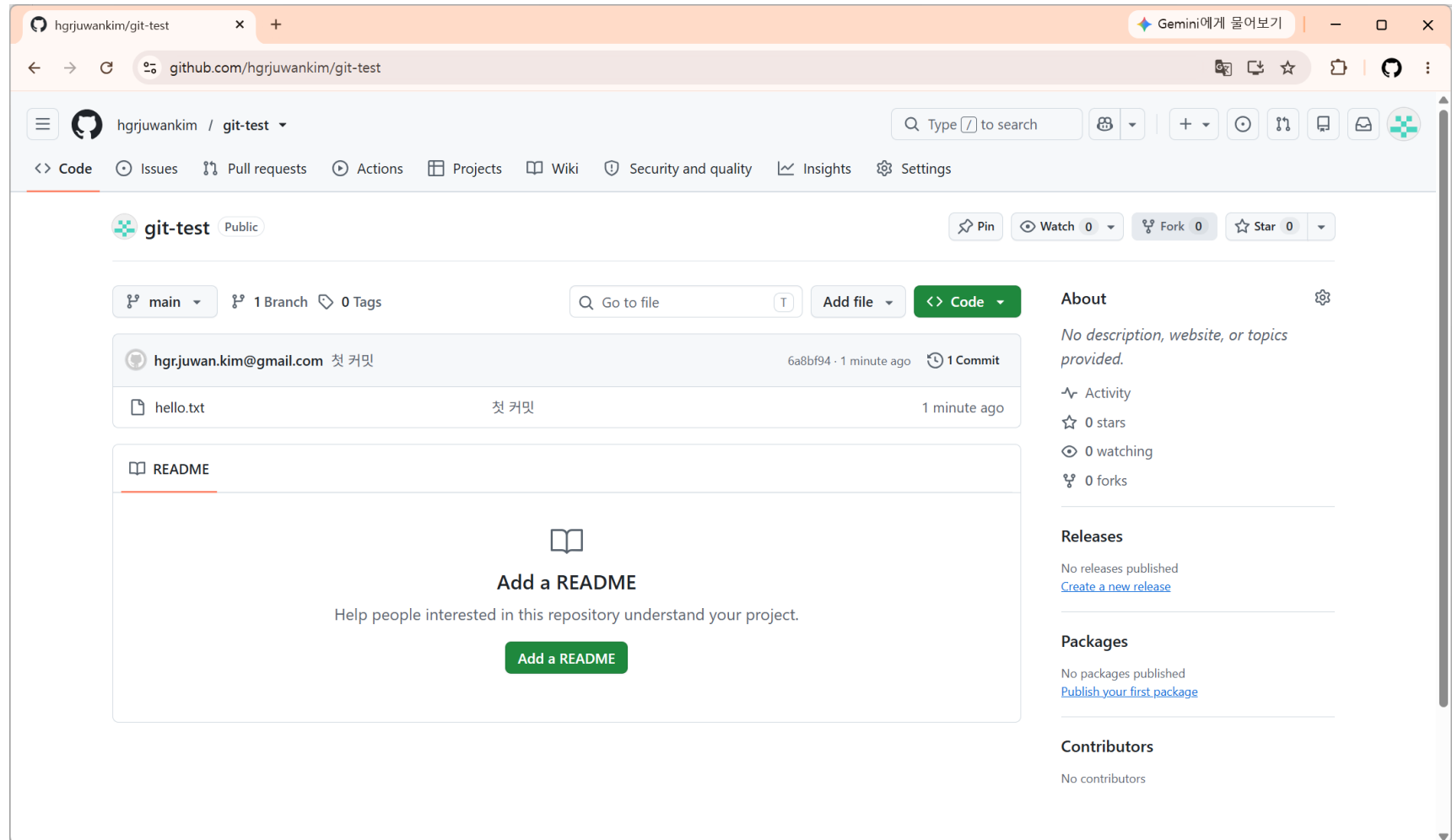
C:\git test>git push -u origin main
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 229 bytes | 229.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/hgrjuwankim/git-test.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.

C:\git test>
```

• GitHub push 결과 확인하기

> 터미널과 GitHub 페이지 양쪽에서 확인

- push 완료 메시지를 확인하고, GitHub 저장소 페이지를 새로고침해 파일이 올라왔는지 확인합니다.



• 클로드에게 커밋 메시지 부탁하기

> 바이브 코딩 팁

- git diff 결과를 클로드에게 보여주고, 변경 내용에 맞는 커밋 메시지를 요청하면 훨씬 편합니다.

```
C:\WINDOWS\system32\cmd.exe
C:\git test>git diff
diff --git a/hello.txt b/hello.txt
index f2aa86d..e4c99d1 100644
--- a/hello.txt
+++ b/hello.txt
@@ -1,2 @@
 hello
+git
\ No newline at end of file

C:\git test>
```

다음은 git diff의 결과이다. commit 메시지를 추천하여 명령어를 작성해줘.

```
"diff --git a/hello.txt b/hello.txt
index f2aa86d..e4c99d1 100644
--- a/hello.txt
+++ b/hello.txt
@@ -1,2 @@
 hello
+git
\ No newline at end of file"
```

+

Sonnet 5 중간



• 그 외 git 기능들

> Git을 유용하게 사용할 수 있는 기능들

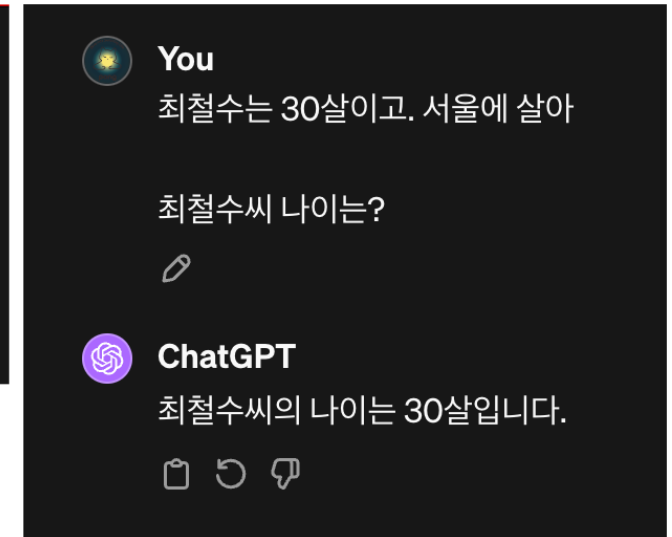
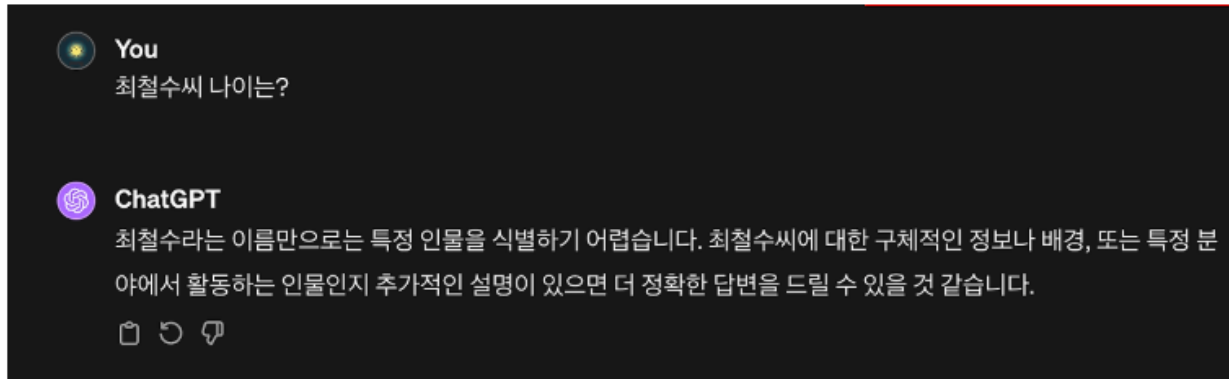
기능	설명
status	변경된 파일, 커밋할 파일 등 현재 작업 상태 를 확인한다.
log	지금까지의 커밋 기록 을 확인한다.
.gitignore	Git이 추적하지 않을 파일·폴더 를 지정하는 설정 파일이다. 이미 추적 중인 파일에는 적용되지 않는다.
clone	원격 저장소를 내 컴퓨터에 복제 한다.
pull	원격 저장소의 변경 내용을 가져와 현재 브랜치에 반영 한다.
branch	독립적인 작업 공간인 브랜치 를 생성·조회·삭제 한다.
checkout	다른 브랜치로 이동 하거나, 파일을 특정 상태로 되돌린다 .
merge	다른 브랜치의 변경 내용을 현재 브랜치에 합친다 .

• RAG 개념

> 대량의 데이터를 학습시킬 수 있는 RAG 사용하기

- RAG(Retrieval-Augmented Generation)란?
 - 검색 증강 생성으로 지정한 텍스트를 데이터베이스로 사용자가 입력한 정보와 연관성이 높은 텍스트를 찾아 프롬프트에 추가함으로 보다 정확한 답변을 할 수 있게 하기 위한 기법

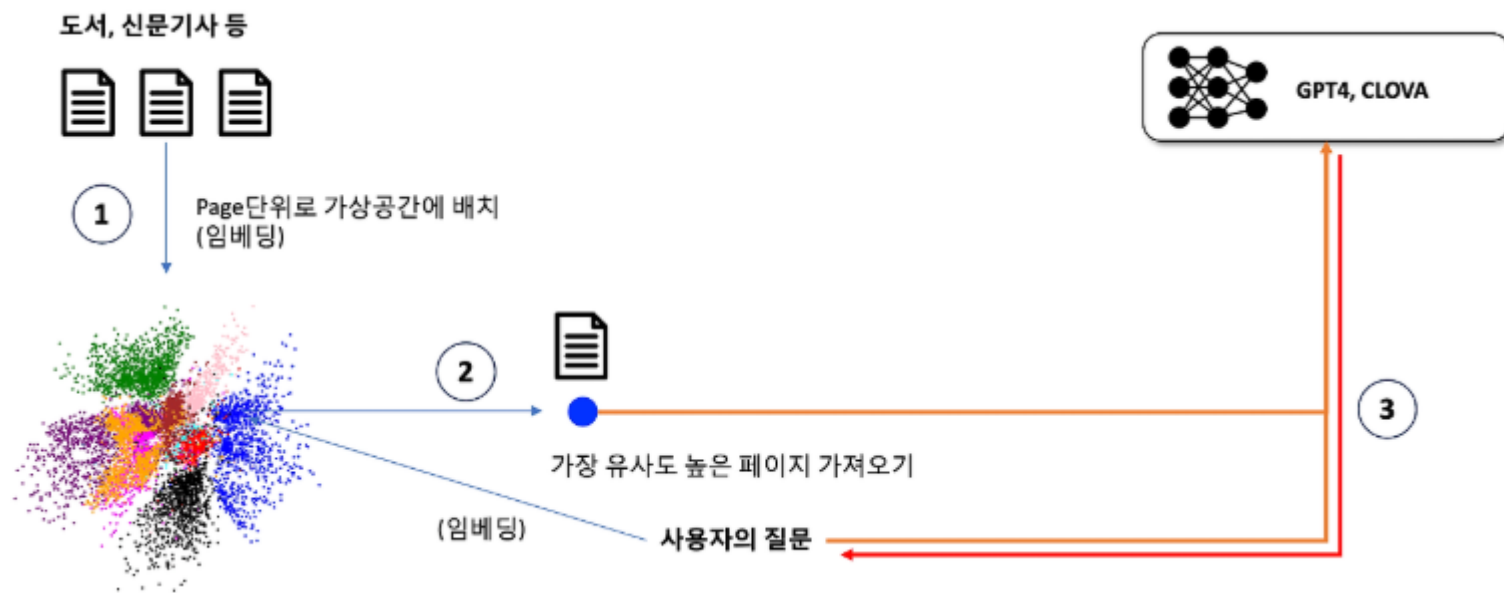
031



• RAG 개념

> 유사한 정보를 가져오는 원리

- 임베딩 기술을 통해 대량의 텍스트를 수학적(벡터)으로 나타내는 방식
- 유사한 단어는 비슷하게 표현하여 연관성을 나타내기 쉽다.
- 이렇게 저장된 정보에서 입력된 값과 연관성이 강한 정보를 찾아 답변에 추가 정보를 넣어 대답한다.



• 환경 준비 결과 확인하기

> 필요 라이브러리 설치

```
pip install -r requirements.txt
```

> 설치가 정상적으로 끝났는지 확인

- 오류 없이 라이브러리 설치가 완료된 터미널 화면을 확인합니다.

```
import openai
import os

openai.api_key = os.environ["OPENAI_API_KEY"]

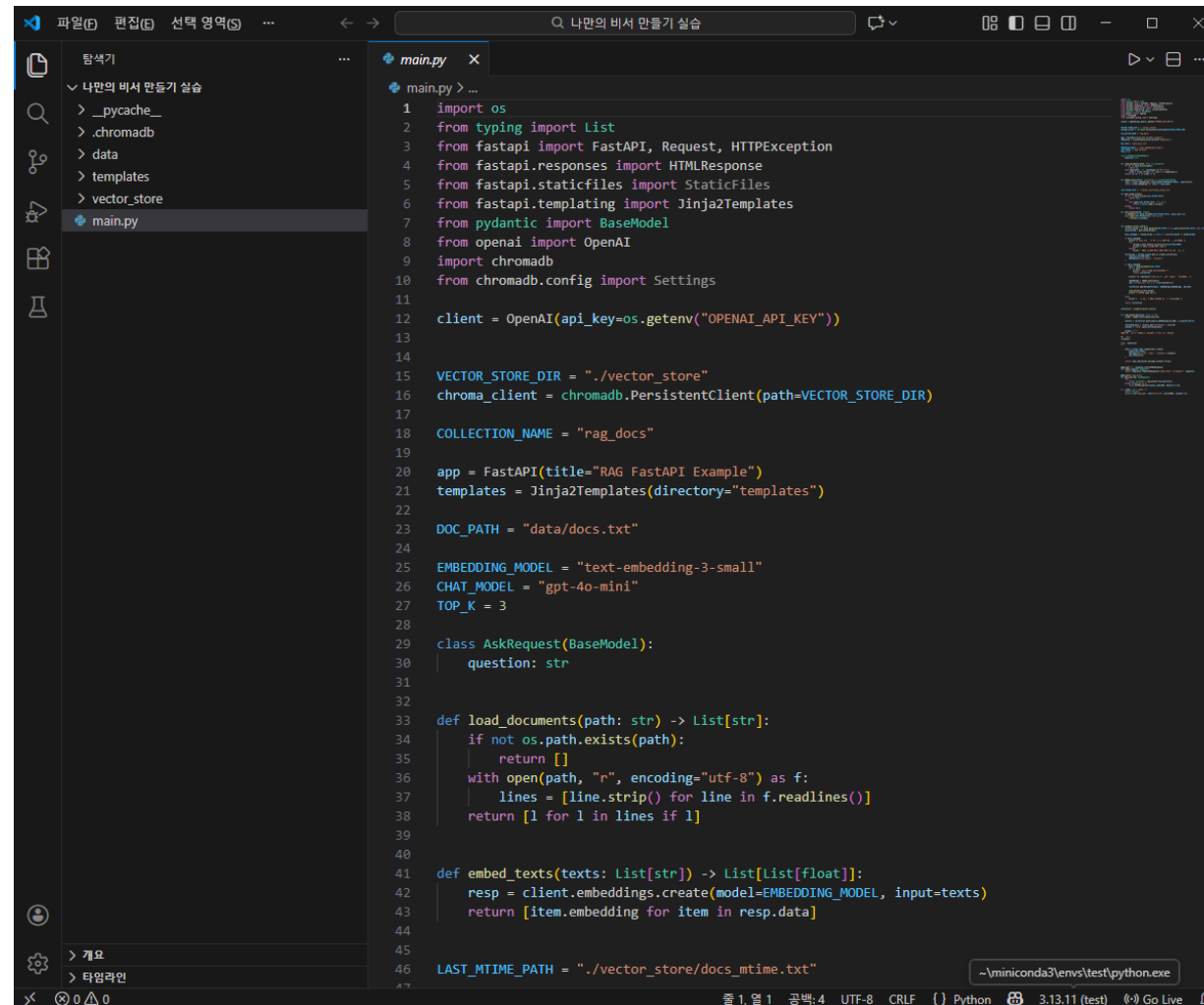
completion = openai.chat.completions.create(
    model="gpt-5-nano",
    messages=[ {"role": "user", "content": "파이썬에 대해 알려주세요."} ] )

print(completion.choices[0].message)
```

- RAG 실습

> RAG 실습 > app.py 실행

034



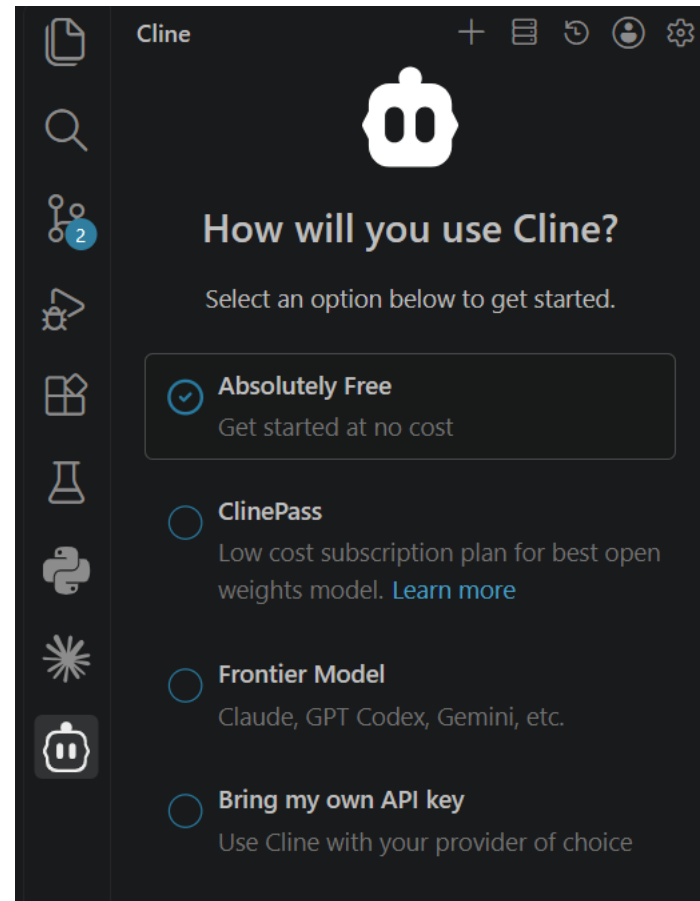
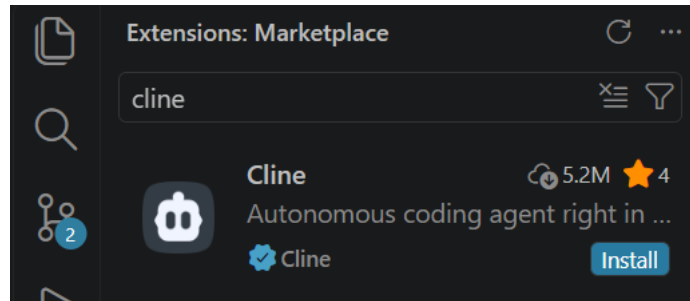
The screenshot shows a code editor with a file explorer on the left and a code editor on the right. The file explorer shows a project structure with folders like `_pycache_`, `.chromadb`, `data`, `templates`, `vector_store`, and a file `main.py`. The code editor displays the contents of `main.py`, which is a FastAPI application for a Retrieval-Augmented Generation (RAG) system. The code includes imports for `os`, `typing`, `fastapi`, `pydantic`, `openai`, and `chromadb`. It defines a `client` for OpenAI, a `chroma_client` for ChromaDB, and a `collection_name` of `"rag_docs"`. The application is configured with a title, templates, and document paths. It defines a `AskRequest` class and a `load_documents` function. The `embed_texts` function is also defined. The application is run using `uvicorn` on port 8000.

```
1 import os
2 from typing import List
3 from fastapi import FastAPI, Request, HTTPException
4 from fastapi.responses import HTMLResponse
5 from fastapi.staticfiles import StaticFiles
6 from fastapi.templating import Jinja2Templates
7 from pydantic import BaseModel
8 from openai import OpenAI
9 import chromadb
10 from chromadb.config import Settings
11
12 client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))
13
14
15 VECTOR_STORE_DIR = "./vector_store"
16 chroma_client = chromadb.PersistentClient(path=VECTOR_STORE_DIR)
17
18 COLLECTION_NAME = "rag_docs"
19
20 app = FastAPI(title="RAG FastAPI Example")
21 templates = Jinja2Templates(directory="templates")
22
23 DOC_PATH = "data/docs.txt"
24
25 EMBEDDING_MODEL = "text-embedding-3-small"
26 CHAT_MODEL = "gpt-4o-mini"
27 TOP_K = 3
28
29 class AskRequest(BaseModel):
30     question: str
31
32
33 def load_documents(path: str) -> List[str]:
34     if not os.path.exists(path):
35         return []
36     with open(path, "r", encoding="utf-8") as f:
37         lines = [line.strip() for line in f.readlines()]
38     return [l for l in lines if l]
39
40
41 def embed_texts(texts: List[str]) -> List[List[float]]:
42     resp = client.embeddings.create(model=EMBEDDING_MODEL, input=texts)
43     return [item.embedding for item in resp.data]
44
45
46 LAST_MTIME_PATH = "./vector_store/docs_mtime.txt"
```

- Cline이란?

- > VS Code 안에서 동작하는 AI 코딩 에이전트

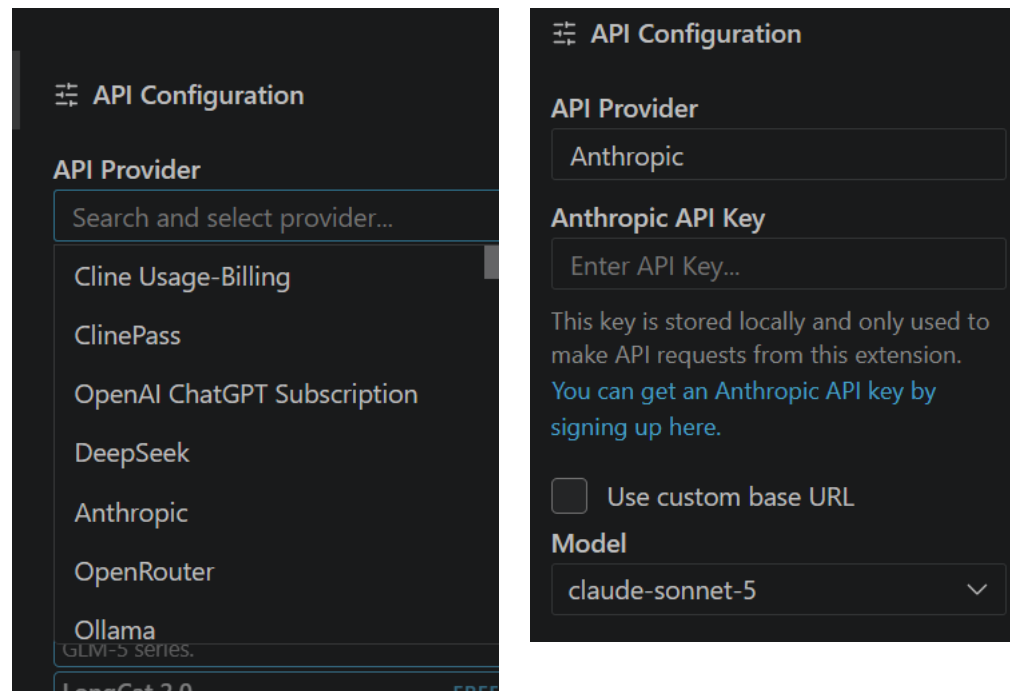
- 요청한 내용에 맞게 AI가 직접 파일을 만들고 수정하고, 필요하면 터미널 명령까지 실행해주는 확장 프로그램입니다.



- Cline에 Claude API 키 연결하기

- > API Provider 설정

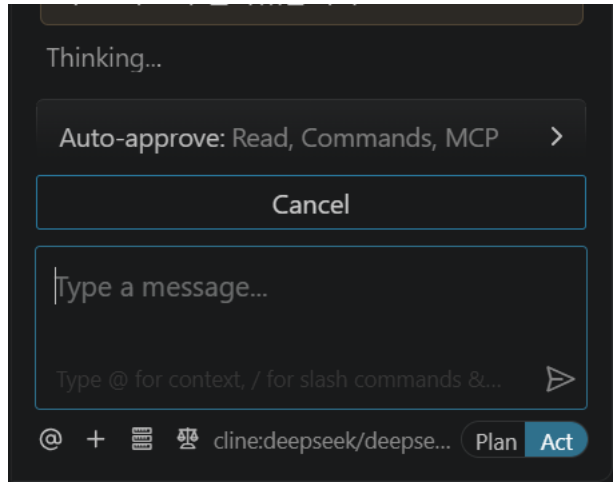
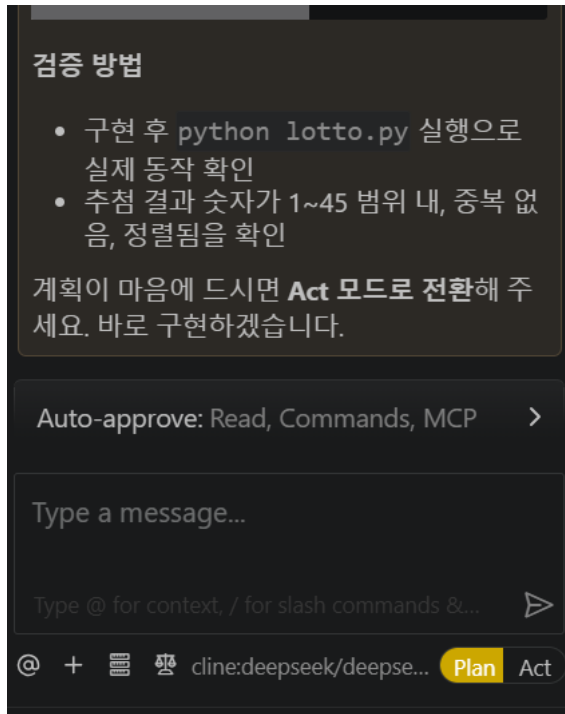
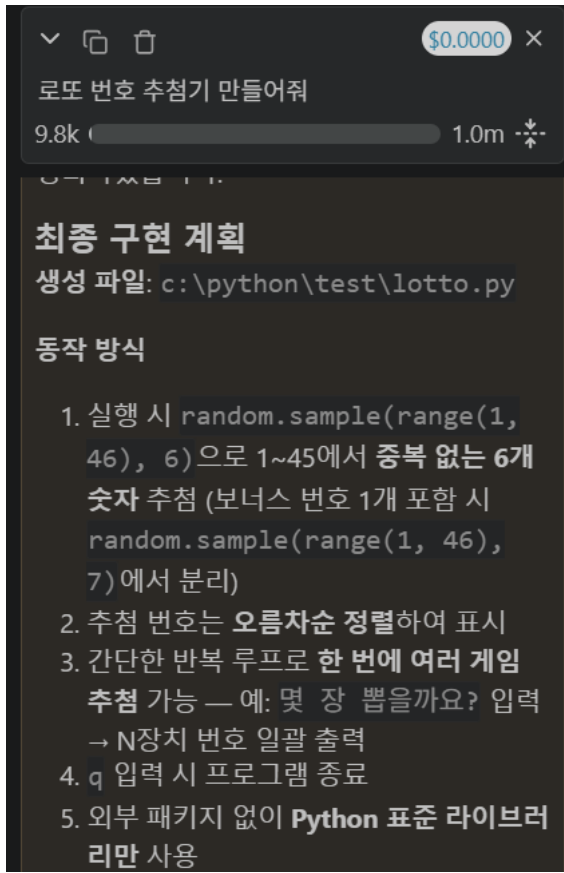
- Cline 설정 화면에서 Anthropic(Claude)을 선택하고, 발급받은 API 키를 입력합니다.



• Cline이 작업하는 과정 살펴보기

> 파일 생성/수정 제안을 확인하고 승인하기

- Cline이 만들려는 파일과 코드 변경 내용을 보여주면, 확인 후 승인(Approve)합니다.



- Cline 사용 팁

- > 작업을 작은 단위로 나누어 요청하기

- 한 번에 너무 큰 기능을 요청하면 오류가 많아집니다.

- > Plan / Act 모드 활용하기

- 먼저 계획(Plan)을 확인하고, 이상이 없으면 실행(Act)하도록 나누어 진행합니다.

- > 결과를 검토한 뒤 커밋하기

- 동작을 확인한 뒤 Git으로 커밋해두면 언제든지 안전하게 되돌릴 수 있습니다.